

BestBite!

Overview

BestBite is a mobile application designed to help NUS students discover free food events based on their proximity.

Telegram groups frequently distribute free food notifications across campus. However, many of these opportunities are not practically useful because students may be located far away from the event location. BiteDance aims to reduce information overload by filtering and presenting only relevant opportunities that are realistically accessible to users.

Ideation

Problem Motivation

NUS students receive numerous free food notifications every day through Telegram channels and group chats.

However:

- Many events are located far from the student's current location.
- Students must manually read and evaluate each notification.
- Important nearby opportunities can easily be missed.
- Constant notifications create unnecessary distractions.

Our goal is to create a location-aware system that helps students efficiently discover nearby free food events while minimizing irrelevant information.

Proposed Core Features

Core Features

- Automatic extraction of free food event information from Telegram groups.
- Parsing and standardization of event details such as time and location.
- Conversion of location names into geographic coordinates.
- Distance-based event filtering.
- Mobile application interface for displaying event information.
- Proximity-based notifications for nearby events.
- Interactive Google Maps integration.
- Visual display of nearby events and walking distances.

Extension Features

Allow users to add specific locations as favourite, so as to track their notifications.

Additional UI beautification.

User Stories

Event Discovery

- As a student, I want free food events to be collected automatically so that I do not need to constantly monitor Telegram groups.

Event Understanding

- As a student, I want event locations and timings to be standardized so that I can quickly understand where and when events are happening.

Personalized Recommendations

- As a student, I want to receive only nearby opportunities instead of every free food notification on campus.

Smart Notifications

- As a student, I want notifications only when events are within a distance that I am willing to travel.

Mobile Accessibility

- As a student, I want all event information to be available through a simple and intuitive mobile application.

Design and Plan

The project follows a pipeline-based architecture:

```
Telegram Groups
  ↓
Message Extraction
  ↓
Event Parsing
  ↓
```

Location Processing
↓
Coordinate Conversion
↓
Distance Calculation
↓
Flutter Frontend
↓
Notification System

Backend Responsibilities

- Connect to Telegram groups.
- Extract incoming event messages.
- Process location and timing information.
- Convert location names into coordinates, then calculate distances using coordinates.
- Prepare event data for frontend consumption.

Frontend Responsibilities

- Display processed event information.
- Present event details clearly to users, including texts, photos and videos.
- Support future location-based recommendations and notifications.

Implemented features:

Feature 1 (Core): Telegram Data Extraction

Description

The system is able to establish a connection with Telegram and automatically retrieve incoming messages from designated groups.

Current Capabilities

- Connects directly to Telegram.
- Automatically captures newly received messages.
- Continuously monitors event-related channels.
- Retrieves event content without manual intervention.

Demo

Type in messages in Telegram chat:



The messages will automatically appear in the run console:

```
C:\Users\blood\Downloads\jdk-21_windows-x64_bin\jdk-21.0.9\bin\java.exe ...  
[main] INFO org.telegram.telegrambots.longpolling.BotSession - Result when calling deleteWebhook: true  
Running...  
It is now functional  
Milestone 1 progress 1/4
```

Feature 2 (Core): Event Parsing and Location Processing

Description

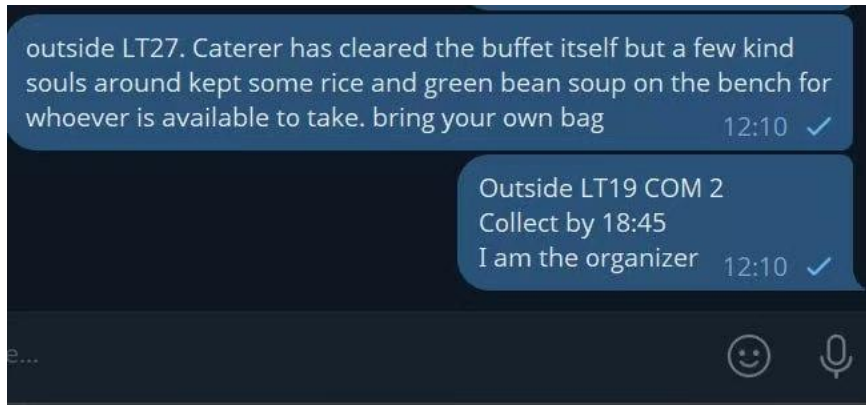
The system extracts important event information from Telegram messages and converts unstructured text into structured data.

Current Capabilities

- Detects location keywords such as:
 - COM3
 - LT27
 - Frontier
 - Other NUS locations
- Detects timing information (as the time when message is sent, will add in the deadline of food event in future milestones)
- Standardizes extracted event details.
- Prepares processed data for further analysis.

Demo

Receiving messages of food events in Telegram chat:



Successfully extract location info from the message and printed in the run console (time information is later displayed in mobile app -> detailed page)

```
Running...
Sent!
LT27
Sent!
LT19 COM 2
```

Feature 3 (Core): Coordinate Conversion

Description

The system converts recognized locations into geographical coordinates.

Current Capabilities

- Maps location names to latitude and longitude values.
- Produces coordinate representations suitable for distance calculations.
- Establishes the foundation for future proximity-based filtering.

Demo

Event locations are converted into geographic coordinates for spatial processing. The Haversine formula is then used to calculate the distance between locations based on their latitude and longitude values. The result of distance between Singapore and Shanghai (3808km) is very similar to the real result (3792km):

```

package org.example;

public class Dist_Calc { new *
    private static double toRadians(double deg) { 4 usages new *
        return deg * Math.PI / 180;
    }

    private static double distance(double lat1, double lon1, double lat2, double lon2) { 2 usages new *
        double dLat = toRadians(deg lat2 - lat1);
        double dLon = toRadians(deg lon2 - lon1);
        double a = Math.pow(Math.sin(dLat / 2), 2) + Math.pow(Math.sin(dLon / 2), 2) * Math.cos(toRadians(lat1)) * Math.cos(toRadians(lat2));
        double theta = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return 6378000 * theta; // using this because the app is for Singapore, which is near the equator. Doesn't really matter though
    }

    public static void main(String[] args) { new *
        System.out.println(distance(lat1: 0, lon1: -90, lat2: 0, lon2: 90));
        System.out.println(distance(lat1: 1.352083, lon1: 103.819836, lat2: 31.22222, lon2: 121.45806)); // SPore <-> SH distance: 3792km
    }
}

```

Feature 4 (Core): Flutter Frontend Integration

Description

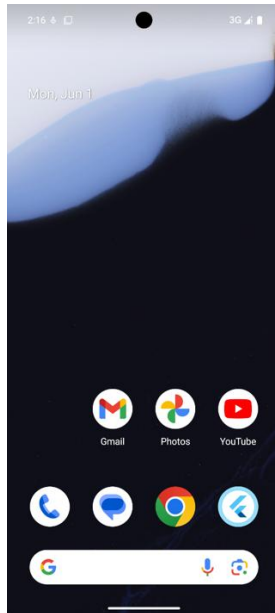
Processed event information can be displayed within the Flutter application.

Current Capabilities

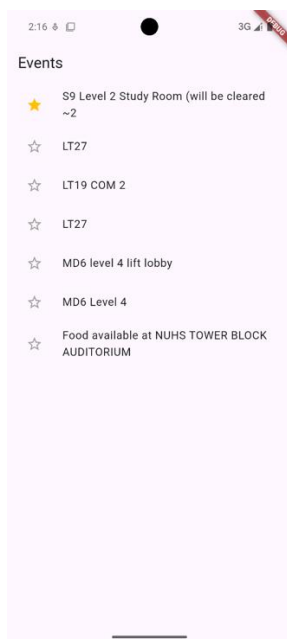
- Event information is transmitted to the frontend.
- Users can view:
 - Event location
 - Event timing
 - Relevant event details
- Provides a basic mobile interface for event browsing.

Demo

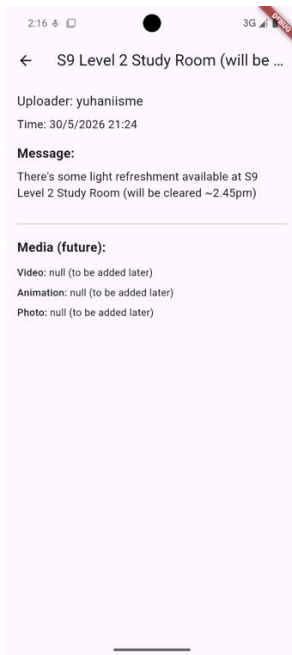
Open up mobile emulator supported by Android Studio:



As the main program is running, event information will automatically appear on the main page of the application:



Users can view detailed events information in detail page (enter the page by clicking the venue):



Planned Features

Feature 5 (Core): User Location Detection

Goal

Obtain the user's real-time location using mobile device GPS.

Planned Functionality

- Real-time GPS access.
- Dynamic location updates.
- Integration with distance calculation modules.

Feature 6 (Core): Filter targeted events based on distance detection

Goal

Filter out the targeted events based on the distance between real time user location and event venue.

Planned Functionality

- Compute user-event distance.
- Filter events outside a specified threshold.
- Prioritize nearby opportunities.

Feature 7 (Core): Proximity-Based Notification System

Goal

Provide personalized notifications based on distance preferences.

Planned Functionality

- Notify users only when relevant events appear nearby.
- Allow customizable travel distances:
 - 300m
 - 500m
 - 1km
 - Custom values

Feature 8: Google Maps Integration

Goal

Provide map-based visualization of free food events.

Planned Functionality

- Interactive map display.
- Allow users to search for locations they are interested in, and add these into “favourites” to further track future food events.
- Distance visualization.
- Navigation support.

Problems Encountered

Telegram Connectivity and Network Restrictions

One of the primary challenges encountered during development involved maintaining reliable access to Telegram services and required dependencies.

Issues Faced

- VPN connectivity instability.
- Difficulty accessing external resources.
- Dependency download interruptions.
- Development environment setup challenges.

Solutions Adopted

- Utilized Alibaba Cloud mirrors to download required dependencies.
- Adjusted development workflow to accommodate network restrictions.

```
allprojects {
    repositories {
        maven { url = uri("https://maven.aliyun.com/repository/google") }
        maven { url = uri("https://maven.aliyun.com/repository/public") }
        google()
        mavenCentral()
    }
}
```

These solutions allowed development to continue despite connectivity limitations.

Future Work

The next stage of development will focus on completing the project's location-aware recommendation pipeline.

Immediate Priorities

- Implement user GPS location detection.
- Develop distance calculation algorithms.
- Introduce event filtering mechanisms.
- Build proximity-based notification services.

Long-Term Improvements

- Google Maps integration.
- Enhanced event visualization.
- Personalized recommendation systems.
- Improved user interface and experience.